

PIP Binary Specification v0.1

A PIP binary file (`.pipb` extension) is formed of a *header* followed by one or more *tables*. The header contains the following fields.

Offset	Size	Value
0×00	4	"PIB\0" (<code>\x50\x49\x42\x00</code>)
0×04	4	Version number: 1
0×08	8	Table count
0×10	8	Interface UID
0×18	8	Checksum: 0 - sum of bytes in rest of header
0×20, 0×30, 0×40, ...	8	Table offsets - offset in bytes from the start of the file
0×28, 0×38, 0×48, ...	8	Table types - values listed below, each corresponds to the preceeding table offset

Table types

Value	Type
0×00	NULL table - ignored by all tooling
0×01	Interface table
002 to 2 ⁶³	Reserved
2 ⁶³	Reserved
2 ⁶³ + 1 to 2 ⁶⁴	Reserved

Interface table

An *interface table* consists of a header, immediately followed by an array of entries, where the index of each entry corresponds to the numerical ID from the source file

Offset from start of header	Size	Value
0×00	8	Entry count
0×08	8	Checksum: 0 - sum of bytes in rest of table including methods

Reserved entry

A reserved entry is 8 bytes of 0.

Function entry

The parameters from the source file are placed into the argument fields as follows.

1. The arguments are categorised into MEMORY (`mem` , `outmem` , `str` , `outstr`) and INTEGER (all other types, including *enumeration* and *bitfield* types).
2. If all arguments are INTEGER, they are placed in definition order into *Argument A*, *Argument Bx*, then *Argument By*.
3. If one argument is MEMORY, it is placed into *Argument Pair B*, and the INTEGER argument is placed in *Argument A*.
4. Any other combination should be illegal if I've done this right.

Each function entry has the following format

Offset from start of method entry	Size	Value
0×00	4	Entry type: 1
0×04	1	Return type (according to "type values" table)
0×05	1	Argument A (according to "type values" table)
0×06	2	Argument Pair B

Argument pair values: (all other values reserved)

Value	Description
0×0101	<code>mem</code> type
0×0102	<code>str</code> type
0×0103	<code>outmem</code> type
0×0104	<code>outstr</code> type
0×02xy	Pair of primitive types, where <code>x</code> is the first type and <code>y</code> is the second type according to the "type values" table

Type values: (all other values reserved)

Value	Description
0×0	Unused argument
0×1	Any integer type
0×2	Object type

Example

```
func@0 write(id: u8);
reserve@1;
func@2 foo(data: str, val: n64) -> !err;
```

Byte	0 to 4 (entry type)	5 (return type)	6 (argument A)	7 to 8 (argument pair B)
0×0X	0×1 (func)	0×0 (unused)	0×1 (integer)	0×0200 (unused, unused)
0×1X	0×0 (reserve)	0×0	0×0	0×0
0×2X	0×1 (func)	0×1 (integer)	0×1 (integer)	0×0102 (str)